Gravity-Bot

Rapport Itération 2

Groupe CD3

Abid Mohamed, El Amrany Anas, El Haddadi Bilele, El Hauat Chada, El Qabbab Oussama, Hosayn Mohamed Anis, Kibou Yassine, Masbah Louay, Samari Yassine

Table des matières

Fonctionnalités ajoutés 3
Boucle de jeu 3
Nouveau obstacles 3
Placement des obstacles 4
UI du menu principal 4
Utilisation de l'IA 4

Fonctionnalités ajoutés

Boucle de jeu

Nous avons implémenté la boucle de jeu. Chaque tour, un joueur peut jouer et placer des obstacles. Au tour suivant, c'est à l'autre joueur de les placer.

L'interface affiche le numéro de tour, le nom du joueur, le budget. Le joueur peut placer des obstacles et le cout sera déduit de son budget. Le budget est un attribut de la classe `Joueur`. Le prix de l'obstacle est un attribut de la classe `Obstacle`.

La boucle de jeu fonctionne à l'aide de 2 classes.

Game : Contient la logique de tour. Il compte le nombre de tour, lance la simulation du tour et vérifie si le joueur à gagner. Dans le cas contraire, il change le joueurCourant. Game ne gère pas l'affichage. C'est une classe du package interface (`FenetreJeu`) qui lit à chaque frame les données de Game.
Il est pas encore possible de gagner, le jeu se termine sans recommencer.

Map : Contient les informations situé dans la map. Les obstacles (dans un HashSet), la position initiale et le vitesse initiale du projectile, la position de la cible. Lorsque `Game` lance la simulation, le projectile retourne à son état initiale.
A chaque frame, la map vérifie que le projectile entre en collision avec un obstacle, si c'est le cas, il exécute `eventAction` de l'obstacle. L'obstacle peut interagir avec le projectile avec la poignée de ce dernier.
La map vérifie aussi à chaque instant que le projectile ne sort pas des limites de la map ou si il a pas atteint la cible. Si c'est le cas, il arrête la simulation et return `true` ou `false` si le tour est gagnant ou perdant.
La map est un `JPanel` et affiche des carrés aux positions du projectile, des obstacles et de la cible. C'est une mesure temporaire pour visualiser le jeu car les `Acteur` ne gère pas encore leur propre affichage.
Pour l'instant, la map est initialisé avec des paramètres par défaut. A terme, il faudrait qu'elle soit construit à l'aide d'un fichier et place les obstacles déjà présent sur la map.

Nouveau obstacles

L'obstacle Propulseur et Téléporteur ont été ajouté. Ils hérite d'un type différent d'obstacle `ObstacleSegment` qui est construit avec 2 points : la position initiale et finale. Ces 2 points permettent de donner la zone d'action. Pour le téléporteur, l'entrée et la sortie. Pour le propulseur, le début et la fin.

Il est pas encore possible des les placer, l'implémentation est en cours. 2 cliques seront nécessaires pour placer ces acteurs.

Placement des obstacles

Le placement des obstacles à été ajouté. Un menu sidebar dans l'interface `FenetreJeu` liste les obstacles. Une fois le bouton cliqué, la classe est instancié. Quand le joueur clique sur la `Map` qui est un `JPanel`, la position de l'obstacle prend la position de la souris, l'obstacle est copié et mis dans la liste d'obstacle de la `Map`.

La fonction copié est une méthode de la classe abstraite `Obstacle`.

Obstacle contient peu d'attribut et beaucoup de constantes (puissance du propulseur, puissance de l'aimant, rayon d'action), c'est par volonté de crée une liste d'obstacles peu importante et unique. Il est donc plus simple de les crée.

UI du menu principal

L'UI du menu principal a été améliorée. Un gif en arrière-plan représente l'espace pour une immersion immédiate. Les boutons ont une épaisseur grâce à une bordure arrondie personnalisée via `Graphics2D` et l'utilisation de l'anticrénélage pour un rendu plus propre et lissé.

Le menu gère l'audio et la navigation à l'aide de plusieurs composants.

MenuPrincipal gère l'affichage de la fenêtre et le passage entre les différents modes. Il utilise un `GridBagLayout` pour centrer les boutons. Au survol, un MouseListener change la couleur du texte pour donner un feedback visuel. Un curseur personnalisé a aussi été ajouté. Lorsqu'on clique sur un bouton, la fenêtre actuelle est détruite avec dispose() pour libérer de la mémoire avant d'ouvrir le mode suivant (Jeu, Éditeur, ou Paramètres).

Le menu est lancé sur un thread séparé avec invokeLater pour éviter les problèmes d'affichage lors du démarrage.

Utilisation de l'IA

L'IA à été utilisé dans ce projet. Tout le code produit par l'IA à été vérifié et approuvé. Sont concerné les tâches suivantes :

- Design de l'interface `MenuPrincipal`. L'IA à rendu le code difficile à lire car beaucoup de lignes concernent le réglage des boutons. Il aurait été préférable de crée les composants et boutons dans d'autres classes pour être réutilisés.
- Documentation de certaines parties du code (`FenetreJeu`, `MenuPrincipal`, `Credit`, `Paramètres`). La génération des documentations suit les règles `JavaDoc` et nous a offert un gain de temps.
- Optimisation du code. Nos compétences nous limites à devoir effectuer une tâche en même temps. Cependant, cela rend l'opération de simulation du projectile bloquante. L'IA nous à proposer d'utiliser un timer et de rafraîchir l'écran quelques fois par

secondes. Notre implémentation robuste avec le `detlaT` à rendu cette opération facile à faire